

CHAPTER 6: CISC AND RISC ARCHITECTURES

PROCESSOR ARCHITECTURES

- This chapter explores the fundamental design philosophies of a **CPU's instruction set architecture (ISA)**.
- The two dominant philosophies are **CISC** (Complex Instruction Set Computer) and **RISC** (Reduced Instruction Set Computer).
- These designs represent different trade-offs between hardware complexity and software complexity.

CISC: COMPLEX INSTRUCTION SET COMPUTER

- **Characteristics:**
 - **Emphasis on hardware.**
 - **Large number of instructions:** Includes many complex, specialized instructions that may take multiple clock cycles to execute.
 - **Variable-length instruction formats:** This can make decoding instructions complex.
 - **Many addressing modes:** Provides flexibility in how memory operands are accessed.
 - **Goal:** Accomplish tasks in as few lines of *assembly code* as possible. A single instruction can perform multi-step operations (e.g., MUL M, R1 could fetch from memory, perform multiplication, and store the result all at once).
- **Examples:** Intel x86 (used in most PCs and laptops), VAX, Motorola 68000.

LIMITATIONS OF CISC

- **Hardware Complexity:** A large instruction set and many addressing modes make the processor hardware (decoder, control unit) complex, larger, and more power-hungry.
- **Performance Issues:** It's difficult to **pipeline** instructions efficiently because they have variable lengths and execution times.
- **Inefficient Use:** Studies showed that compilers and programmers often used only a small subset of the available simple instructions, ignoring the complex ones.

RISC: REDUCED INSTRUCTION SET COMPUTER

- **Features:**
 - **Emphasis on software.**

- **Small, highly optimized set of instructions:** Instructions are simple and execute in a **single clock cycle**.
 - **Fixed-length instruction formats:** Simplifies instruction decoding and pipelining.
 - **Load/Store Architecture:** Only LOAD and STORE instructions access memory. All other operations (like ADD, SUB) work *only* on **CPU registers**.
 - **Large number of registers:** To support the load/store model and reduce memory traffic.
 - **Goal:** Achieve higher performance by making the hardware simpler and faster, allowing for higher clock speeds and efficient pipelining. This shifts the complexity to the **compiler**, which must string together simple instructions to perform complex tasks.
- **Examples:** ARM (used in virtually all smartphones and tablets), MIPS, RISC-V.

VLIW AND EPIC EXPLAINED

- **VLIW (Very Long Instruction Word):**
 - A design philosophy that aims to achieve instruction-level parallelism.
 - The **compiler** (not the hardware) identifies and bundles multiple *independent* instructions (e.g., an integer add, a floating-point multiply, a memory load) into a single, very long "instruction word."
 - The hardware is simple: it just reads this bundle and dispatches each instruction to a corresponding execution unit simultaneously.
- **EPIC (Explicitly Parallel Instruction Computing):**
 - A successor to VLIW, most famously used in Intel's Itanium architecture.
 - Like VLIW, it relies on the compiler to explicitly bundle instructions for parallel execution.
 - It added features for more flexibility, such as hints to the hardware about which instructions can be run in parallel, to overcome some of VLIW's rigidity.

LOGICAL VS. PHYSICAL ADDRESSES

LOGICAL ADDRESS (OR VIRTUAL ADDRESS):

- The address **generated by the CPU** and seen by the program.
- It exists in a theoretical *logical address space* for each process.
- This allows a program to behave as if it has a large, private, contiguous block of memory, even if it doesn't.

PHYSICAL ADDRESS:

- The **actual address** that is sent to the **main memory (RAM)** hardware.

- It corresponds to a physical location on the memory chips.
- **Translation:** The **Memory Management Unit (MMU)**, a piece of hardware, is responsible for translating logical addresses into physical addresses on the fly.

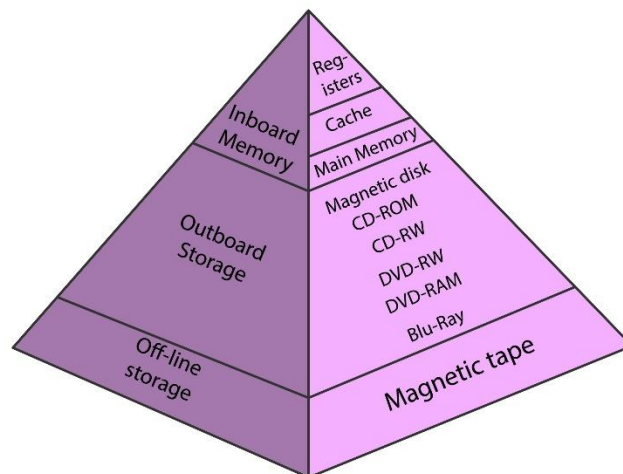
CACHE VS. VIRTUAL MEMORY

Both are memory-management techniques that use a fast/small storage to "pretend" a slow/large storage is faster, but they have different goals:

Feature	Cache Memory	Virtual Memory
Primary Goal	To speed up average memory access time.	To increase the apparent size of main memory.
Components	Fast SRAM (Cache) and slower DRAM (Main Memory).	Fast DRAM (Main Memory) and slower Disk (Storage).
Management	Managed by Hardware (CPU cache controller).	Managed by the Operating System (OS) and Hardware (MMU) .
Speed Mismatch	Solves the CPU vs. RAM speed gap.	Solves the RAM vs. Disk size/speed gap.
Unit of Transfer	Cache Line (e.g., 64 bytes).	Page (e.g., 4 KB).

Diagram: Memory Hierarchy

The memory hierarchy organizes storage into a "pyramid" based on speed, cost, and capacity.



- **Top (Level 0): CPU Registers** (Fastest, smallest, most expensive per bit)
- **Level 1: L1 Cache** (SRAM)
- **Level 2: L2 Cache** (SRAM)
- **Level 3: Main Memory** (DRAM)
- **Level 4: Secondary Storage** (SSD / Hard Drive)
- **Bottom (Level 5): Tertiary Storage** (Tape / Cloud) (Slowest, largest, cheapest per bit)

Principle: As you go *down* the pyramid, capacity **increases**, cost per bit **decreases**, and access speed **decreases**.